

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>练习集创建</title>
  <style>

    /* 整页布局 */
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      margin: 0;
      padding: 20px;
      background-color: #f4f7f6;
      color: #333;
    }

    .container {
      max-width: 1200px;
      margin: 0 auto;
      background-color: #fff;
      padding: 30px;
      border-radius: 8px;
      box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
    }

    h2 {
      text-align: center;
      color: #0056b3;

      margin-bottom: 30px;
      font-size: 2em;
      position: relative;
      padding-bottom: 10px;
    }

    h2::after {
      content: "";
      position: absolute;
      left: 50%;
      bottom: 0;
      transform: translateX(-50%);
      width: 80px;
      height: 4px;
      background-color: #007bff;
      border-radius: 2px;
    }
  </style>
</head>
<body>
  <div class="container">
    <h2>练习集创建</h2>
  </div>
</body>
</html>
```

```
/* 表单部分 */
.form-section {
    margin-bottom: 40px;
    padding: 20px 0;
    border-bottom: 1px dashed #eee;
}

.form-section:last-of-type {
    border-bottom: none;
}

.form-group {
    margin-bottom: 20px;
    display: flex;
    align-items: center;

    flex-wrap: wrap;
}

.form-group label {
    flex: 0 0 120px; /* 标签固定宽度 */
    margin-right: 15px;
    font-weight: bold;
    color: #555;
    text-align: right;
    padding-top: 5px;
    box-sizing: border-box;
}

.form-group .form-control {
    flex: 1; /* 输入框占据剩余空间 */

    padding: 10px 12px;
    border: 1px solid #ddd;
    border-radius: 5px;
    font-size: 1em;
    box-sizing: border-box;
    min-width: 200px; /* 最小宽度限制 */
}

.form-group textarea.form-control {
    min-height: 80px;
    resize: vertical;
    line-height: 1.5;
}

.form-group select.form-control {
    appearance: none; /* 移除默认样式 */
```

## 动车组乘务综合学习训练平台 V1.0

```
.4%22%20height%3D%22292.4%22%3E%3Cpath%20fill%3D%22%23007bff%22%20d%3D%22M287%2069.4a17.6%2017.6%200%200%200-24.8%200L146.2%20186.1%2034.4%2069.4a17.6%2017.6%200%200%200-24.8%2024.8L109.8%20109.8c6.8%206.7%2017.9%206.7%2024.7%200L109.8-109.8a17.6%2017.6%200%200%200%200-24.8z%22%2F%3E%3C%2Fsvg%3E');  
US-  
ASCII,%3Csvg%20xmlns%3D%22http%3A%2F%2Fwww.w3.org%2F2000%2Fsvg%22%20width%3D%22292
```

```
.4%22%20height%3D%22292.4%22%3E%3Cpath%20fill%3D%22%23007bff%22%20d%3D%22M287%2069.4a17.6%2017.6%200%200%200-24.8%200L146.2%20186.1%2034.4%2069.4a17.6%2017.6%200%200%200-24.8%2024.8L109.8%20109.8c6.8%206.7%2017.9%206.7%2024.7%200L109.8-109.8a17.6%2017.6%200%200%200%200-24.8z%22%2F%3E%3C%2Fsvg%3E');  
background-repeat: no-repeat;  
background-position: right 10px center;  
background-size: 12px;  
  
padding-right: 30px; /* 留出箭头位置 */  
}  
  
/* 题目选择区域 */  
.question-selection {  
    background-color: #fbfbfb;  
    border: 1px solid #e0e0e0;  
    border-radius: 6px;  
    padding: 25px;  
    margin-bottom: 40px;  
}  
  
.question-selection h3 {  
    color: #007bff;  
    margin-top: 0;  
    margin-bottom: 20px;  
    border-bottom: 2px solid #e9ecef;  
    padding-bottom: 10px;  
}  
  
.question-filter {  
    display: flex;  
    gap: 15px;  
    margin-bottom: 25px;  
    flex-wrap: wrap;  
}  
  
.question-filter .filter-group {  
    flex: 1;  
    min-width: 180px;  
}
```

```
.question-filter label {
  display: block;
  margin-bottom: 8px;
  font-weight: bold;

  color: #666;
  font-size: 0.9em;
}

.question-list {
  max-height: 400px;
  overflow-y: auto;
  border: 1px solid #e0e0e0;
  border-radius: 5px;
  background-color: #fff;
}

.question-item {
  padding: 12px 15px;
  border-bottom: 1px solid #eee;
  display: flex;
  align-items: center;
  gap: 10px;
  transition: background-color 0.2s ease;
}

.question-item:last-child {
  border-bottom: none;
}

.question-item:hover {
  background-color: #f2f7fc;
}

.question-item input[type="checkbox"] {
  transform: scale(1.2);
  margin-right: 5px;
  cursor: pointer;
}

.question-item span {
  flex: 1;
  font-size: 0.95em;
  color: #444;
}

.selected-questions-summary {
  margin-top: 25px;
  padding: 15px;
```

```
        background-color: #e9f5ff;
        border: 1px solid #cce5ff;
        border-radius: 5px;
        color: #004085;
        font-size: 0.95em;
    }

    /* 提交按钮 */
    .button-group {
        text-align: center;
        margin-top: 40px;
        padding-top: 20px;
        border-top: 1px dashed #eee;
    }

    .button-group button {
        padding: 12px 25px;
        font-size: 1.1em;
        border: none;
        border-radius: 6px;
        cursor: pointer;
        transition: background-color 0.3s ease, transform 0.2s ease;
        margin: 0 10px;
        min-width: 120px;
    }

    .button-group .submit-btn {

        background-color: #007bff;
        color: white;
    }

    .button-group .submit-btn:hover {

        background-color: #0056b3;
        transform: translateY(-2px);
        box-shadow: 0 4px 8px rgba(0, 123, 255, 0.3);
    }

    .button-group .cancel-btn {
        background-color: #f0f2f5;
        color: #666;
        border: 1px solid #ccc;
    }

    .button-group .cancel-btn:hover {
        background-color: #e0e4eb;
        transform: translateY(-2px);
        box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
    }
```

```
}

/* 响应式调整 */
@media (max-width: 768px) {
  .form-group {
    flex-direction: column;
    align-items: flex-start;

  }

  .form-group label {
    width: 100%;
    text-align: left;

    margin-bottom: 8px;

    margin-right: 0;
  }

  .form-group .form-control {
    width: 100%;
    min-width: unset;
  }

  .question-filter {
    flex-direction: column;
  }

  .button-group button {
    width: 100%;
    margin-bottom: 15px;
  }
}
</style>
</head>
<body>
  <div class="container">
    <!-- 页面标题 -->
    <h2>创建练习集</h2>

    <!-- 练习集基本信息 -->
    <div class="form-section">
      <h3>基本信息</h3>
      <div class="form-group">
        <label for="practiceSetName">练习集名称:</label>
        <input type="text" id="practiceSetName" name="practiceSetName" class=
"form-control" placeholder="请输入练习集名称" required>
      </div>

      <div class="form-group">
```

```
<label for="subjectArea">所属科目:</label>
<select id="subjectArea" name="subjectArea" class="form-control"
required>
  <option value="">请选择科目</option>
  <option value="安全规章">安全规章</option>

  <option value="技术操作">技术操作</option>
  <option value="应急处置">应急处置</option>
  <option value="服务礼仪">服务礼仪</option>
</select>
</div>
<div class="form-group">
  <label for="difficultyLevel">难度等级:</label>
  <select id="difficultyLevel" name="difficultyLevel" class=
"form-control" required>
    <option value="">请选择难度</option>
    <option value="初级">初级</option>
    <option value="中级">中级</option>
    <option value="高级">高级</option>
  </select>
</div>
<div class="form-group">
  <label for="description">练习集描述:</label>
  <textarea id="description" name="description" class="form-control"
rows="4" placeholder="请输入练习集的相关描述"></textarea>
</div>
</div>

<!-- 题目选择区域 -->
<div class="question-selection">
  <h3>选择题目</h3>
  <div class="question-filter">
    <div class="filter-group">
      <label for="questionTypeFilter">题目类型:</label>

      <select id="questionTypeFilter" class="form-control">
        <option value="">所有类型</option>
        <option value="单选题">单选题</option>
        <option value="多选题">多选题</option>
        <option value="判断题">判断题</option>
      </select>
    </div>
    <div class="filter-group">
      <label for="questionCategoryFilter">题目类别:</label>
      <select id="questionCategoryFilter" class="form-control">
        <option value="">所有类别</option>

        <option value="理论知识">理论知识</option>
        <option value="实操技能">实操技能</option>
      </select>
    </div>
  </div>
</div>
```

```
        <option value="案例分析">案例分析</option>
    </select>
</div>
<div class="filter-group">
    <label for="questionKeywordFilter">关键词搜索:</label>
    <input type="text" id="questionKeywordFilter" class=
"form-control" placeholder="请输入关键词">
</div>
</div>

<!-- 题目列表 -->
<div class="question-list" id="questionList">
    <!-- 以下是模拟的题目数据，实际应通过 JS 动态加载 -->
    <div class="question-item">
        <input type="checkbox" id="q1" value="q1">
        <span>(单选题) 列车在紧急制动时，乘务员应首先向哪个方向疏散旅客？</span>
    </div>
    <div class="question-item">
        <input type="checkbox" id="q2" value="q2">
        <span>(判断题) 动车组运行中发现车厢异味时，应立即进行排风处理。</span>
    </div>
    <div class="question-item">
        <input type="checkbox" id="q3" value="q3">
        <span>(多选题) 动车组停站开车前，乘务员需要检查哪些事项？</span>
    </div>

    <div class="question-item">
        <input type="checkbox" id="q4" value="q4">
        <span>(单选题) 《铁路旅客运输规程》规定，儿童身高超过多少厘米需购买儿童票？</span>
    </div>
    <div class="question-item">
        <input type="checkbox" id="q5" value="q5">
        <span>(判断题) 乘客在车厢内吸烟是允许的，但必须在指定区域。</span>
    </div>
    <div class="question-item">
        <input type="checkbox" id="q6" value="q6">
        <span>(单选题) 在动车组列车上，向旅客提供热饮的规定温度范围是多少？</span>
    </div>

    <div class="question-item">
        <input type="checkbox" id="q7" value="q7">
        <span>(多选题) 办理旅客退票时，车站或代售点需要检查哪些证件信息？</span>
    </div>
    <div class="question-item">
        <input type="checkbox" id="q8" value="q8">
        <span>(判断题) 旅客携带宠物乘坐动车组列车时，必须将其装入专用笼具。</span>
    </div>
    <div class="question-item">
        <input type="checkbox" id="q9" value="q9">
        <span>(单选题) 遇突发事件，乘务员使用列车紧急制动装置的正确步骤是？</span>
    </div>
</div>
```



```
</div>
<div class="question-item">
  <input type="checkbox" id="q10" value="q10">
  <span>(多选题) 动车组列车服务过程中, 保障旅客饮水安全的主要措施有哪些? </span>
</div>
<div class="question-item">
  <input type="checkbox" id="q11" value="q11">
  <span>(判断题) 损坏列车设施设备, 应立即报告列车长并承担相应责任。</span>
</div>
<div class="question-item">
  <input type="checkbox" id="q12" value="q12">
  <span>(单选题) 高铁列车在隧道内运行时, 车窗玻璃可能出现短暂的何种现象? </span>
</div>
<div class="question-item">
  <input type="checkbox" id="q13" value="q13">
  <span>(多选题) 动车组餐车服务人员在提供餐饮服务时应注意哪些卫生规范? </span>
</div>
<div class="question-item">
  <input type="checkbox" id="q14" value="q14">
  <span>(判断题) 动车组列车洗手间的清洁工作应每隔两小时进行一次。</span>
</div>
<div class="question-item">
  <input type="checkbox" id="q15" value="q15">
  <span>(单选题) 动车组列车乘务员的日常工作内容不包括以下哪项? </span>
</div>
<div class="question-item">
  <input type="checkbox" id="q16" value="q16">
  <span>(多选题) 铁路部门在保障春运期间旅客出行方面通常采取哪些措施? </span>
</div>
<div class="question-item">
  <input type="checkbox" id="q17" value="q17">
  <span>(判断题) 列车在运行过程中, 任何人都可以随意按动紧急停车按钮。</span>
</div>
<div class="question-item">
  <input type="checkbox" id="q18" value="q18">
  <span>(单选题) 动车组列车停靠站台时, 车门关闭前乘务员应进行什么确认? </span>
</div>
<div class="question-item">
  <input type="checkbox" id="q19" value="q19">
  <span>(多选题) 遇到霸座行为, 乘务员可以采取哪些措施进行处理? </span>
</div>
<div class="question-item">
  <input type="checkbox" id="q20" value="q20">
  <span>(判断题) 遗失物品的旅客可在列车到站后 24 小时内联系客服找回。</span>
</div>
</div>

<div class="selected-questions-summary" id="selectedQuestionsSummary">
</div>
</div>
```

```
<!-- 操作按钮 -->
<div class="button-group">
  <button type="submit" class="submit-btn">创建练习集</button>
  <button type="reset" class="cancel-btn">取消</button>
</div>
</div>

<script>
  // 获取页面元素
  const questionList = document.getElementById('questionList');
  const selectedQuestionsSummary =
document.getElementById('selectedQuestionsSummary');
  const questionTypeFilter = document.getElementById('questionTypeFilter');
  const questionCategoryFilter =
document.getElementById('questionCategoryFilter');
  const questionKeywordFilter =
document.getElementById('questionKeywordFilter');
  const allQuestionItems = questionList.querySelectorAll('.question-item'); //
获取所有题目项

  // 更新选中题目数量的函数
  /**
   * 更新已选中题目数量的显示
   */
  function updateSelectedQuestionCount() {
    // 获取所有被选中的题目复选框
    const selectedCount = questionList.querySelectorAll('input[type=
"checkbox"]:checked').length;
    selectedQuestionsSummary.textContent = `已选择 ${selectedCount} 道题目。`;
  }

  // 初始化时更新一次数量
  updateSelectedQuestionCount();

  // 为每个复选框添加事件监听器
  allQuestionItems.forEach(item => {
    const checkbox = item.querySelector('input[type="checkbox"]');
    checkbox.addEventListener('change', updateSelectedQuestionCount);
  });

  // 筛选功能
  /**
   * 根据筛选条件过滤题目列表
   */
  function filterQuestions() {
    const typeValue = questionTypeFilter.value;
    const categoryValue = questionCategoryFilter.value;
    const keywordValue = questionKeywordFilter.value.toLowerCase();
```

## 动车组乘务综合学习训练平台 V1.0

```
        allQuestionItems.forEach(item => {
            const textContent =
item.querySelector('span').textContent.toLowerCase();
            // 检查题目类型是否匹配
            const matchesType = typeValue === '' ||
textContent.includes(`${typeValue.toLowerCase()}`);
            // 检查题目类别（模拟，实际需要题目数据中包含类别信息才能准确筛选）

            // 这里简单地假设关键词中可能包含类别信息
            const matchesCategory = categoryValue === '' ||
textContent.includes(categoryValue.toLowerCase());
            // 检查关键词是否匹配
            const matchesKeyword = keywordValue === '' ||
textContent.includes(keywordValue);

            // 根据所有条件判断是否显示题目
            if (matchesType && matchesCategory && matchesKeyword) {
                item.style.display = 'flex'; // 显示题目
            } else {
                item.style.display = 'none'; // 隐藏题目
            }
        });
    });

    // 为筛选条件添加事件监听器
    questionTypeFilter.addEventListener('change', filterQuestions);
    questionCategoryFilter.addEventListener('change', filterQuestions);
    questionKeywordFilter.addEventListener('input', filterQuestions); // 输入时即时筛选

</script>

</body>
</html>
```

// 练习集创建模块

```
/**
 * 训练平台数据结构定义
 */
const platformDataStructures = {
    // 试题类型枚举
    questionTypes: {
        SINGLE_CHOICE: 'singleChoice', // 单选题
```

## 动车组乘务综合学习训练平台 V1.0

```
MULTIPLE_CHOICE: 'multipleChoice', // 多选题
TRUE_FALSE: 'trueFalse', // 判断题
FILL_IN_BLANK: 'fillInBlank', // 填空题
SHORT_ANSWER: 'shortAnswer', // 简答题
},

// 试题难度枚举

questionDifficulties: {
  EASY: 'easy', // 简单
  MEDIUM: 'medium', // 中等
  HARD: 'hard', // 困难
},

// 知识点分类
knowledgeCategories: [
  { id: 'drivingKnowledge', name: '驾驶知识' },

  { id: 'serviceRegulations', name: '服务规范' },
  { id: 'emergencyHandling', name: '应急处置' },
  { id: 'vehicleStructure', name: '车辆结构' },
  { id: 'safetyRegulations', name: '安全规章' },

  { id: 'passengerComm', name: '旅客沟通' },
  // 更多分类
],
};

/**
 * 练习集数据模型

 * @param {string} setId - 练习集唯一标识
 * @param {string} setName - 练习集名称
 * @param {string} setDescription - 练习集描述
 * @param {Array<string>} knowledgePoints - 关联的知识点 ID 列表
 */
function ExerciseSet(setId, setName, setDescription, knowledgePoints = []) {
  this.setId = setId; // 练习集标识
  this.setName = setName; // 练习集名称
  this.setDescription = setDescription; // 练习集描述
  this.knowledgePoints = knowledgePoints; // 关联知识点
  this.questions = []; // 包含的试题列表
  this.createdAt = new Date(); // 创建时间
  this.updatedAt = new Date(); // 更新时间
}

/**
 * 试题数据模型
 * @param {string} questionId - 试题唯一标识
 * @param {string} type - 试题类型
```

## 动车组乘务综合学习训练平台 V1.0

```
* @param {string} content - 试题内容
* @param {string} difficulty - 试题难度
* @param {Array<string>} relatedKnowledgePoints - 关联的知识点 ID 列表
*/
function QuestionItem(questionId, type, content, difficulty,
relatedKnowledgePoints = []) {
  this.questionId = questionId; // 试题标识
  this.type = type; // 试题类型
  this.content = content; // 试题内容
  this.options = []; // 选项列表 (单选/多选/判断)
  this.correctAnswer = null; // 正确答案 (单选/判断/填空/简答)
  this.difficulty = difficulty; // 难度等级
  this.relatedKnowledgePoints = relatedKnowledgePoints; // 关联知识点
  this.analysis = ''; // 答案解析
  this.createdAt = new Date(); // 创建时间
  this.updatedAt = new Date(); // 更新时间
}

/**
 * 单选题模型
 * @param {string} questionId - 试题唯一标识
 * @param {string} content - 试题内容
 * @param {Array<string>} options - 选项列表
 * @param {string} correctAnswer - 正确选项
 * @param {string} difficulty - 难度
 * @param {Array<string>} relatedKnowledgePoints - 关联知识点
 */
function SingleChoiceQuestion(questionId, content, options, correctAnswer,
difficulty, relatedKnowledgePoints) {
  QuestionItem.call(this, questionId, platformDataStructures.questionTypes.
SINGLE_CHOICE, content, difficulty, relatedKnowledgePoints);
  this.options = options; // 选项列表
  this.correctAnswer = correctAnswer; // 正确答案
}

/**
 * 多选题模型
 * @param {string} questionId - 试题唯一标识
 * @param {string} content - 试题内容
 * @param {Array<string>} options - 选项列表
 * @param {Array<string>} correctAnswers - 正确选项列表
 * @param {string} difficulty - 难度
 * @param {Array<string>} relatedKnowledgePoints - 关联知识点
 */
function MultipleChoiceQuestion(questionId, content, options, correctAnswers,
difficulty, relatedKnowledgePoints) {
  QuestionItem.call(this, questionId, platformDataStructures.questionTypes.
MULTIPLE_CHOICE, content, difficulty, relatedKnowledgePoints);
```

## 动车组乘务综合学习训练平台 V1.0

```
this.options = options; // 选项列表
this.correctAnswer = correctAnswers; // 正确答案 (数组)
}

/**
 * 判断题模型
 * @param {string} questionId - 试题唯一标识
 * @param {string} content - 试题内容
 * @param {boolean} correctAnswer - 正确答案 (true/false)
 * @param {string} difficulty - 难度
 * @param {Array<string>} relatedKnowledgePoints - 关联知识点
 */
function TrueFalseQuestion(questionId, content, correctAnswer, difficulty,
relatedKnowledgePoints) {
    QuestionItem.call(this, questionId, platformDataStructures.questionTypes.
TRUE_FALSE, content, difficulty, relatedKnowledgePoints);
    this.correctAnswer = correctAnswer; // 正确答案
}

/**
 * 填空题模型
 * @param {string} questionId - 试题唯一标识
 * @param {string} content - 试题内容 (包含占位符, 例如 "__")
 * @param {Array<string>} correctAnswers - 正确答案列表 (对应占位符)
 * @param {string} difficulty - 难度
 * @param {Array<string>} relatedKnowledgePoints - 关联知识点
 */
function FillInBlankQuestion(questionId, content, correctAnswers, difficulty,
relatedKnowledgePoints) {
    QuestionItem.call(this, questionId, platformDataStructures.questionTypes.
FILL_IN_BLANK, content, difficulty, relatedKnowledgePoints);
    this.correctAnswer = correctAnswers; // 正确答案
}

/**
 * 简答题模型
 * @param {string} questionId - 试题唯一标识
 * @param {string} content - 试题内容
 * @param {string} correctAnswer - 参考答案
 * @param {string} difficulty - 难度
 * @param {Array<string>} relatedKnowledgePoints - 关联知识点
 */
function ShortAnswerQuestion(questionId, content, correctAnswer, difficulty,
relatedKnowledgePoints) {
    QuestionItem.call(this, questionId, platformDataStructures.questionTypes.
SHORT_ANSWER, content, difficulty, relatedKnowledgePoints);
    this.correctAnswer = correctAnswer; // 参考答案
}

/**
```

## 动车组乘务综合学习训练平台 V1.0

```
* 练习集管理服务
*/
const exerciseSetService = {

  exerciseSets: [], // 所有练习集
  questionsDataStore: [], // 所有试题暂存区，实际应从数据库加载

  /**
   * 生成唯一 ID
   * @returns {string} 唯一标识
   */
  generateUniqueId: function() {
    return 'id_' + Date.now() + '_' + Math.random().toString(36).substr(2, 9);
  },

  /**
   * 创建新的练习集
   * @param {string} setName - 练习集名称

   * @param {string} setDescription - 练习集描述
   * @param {Array<string>} knowledgePoints - 关联的知识点 ID 列表
   * @returns {ExerciseSet} 创建的练习集对象
   */
  createExerciseSet: function(setName, setDescription, knowledgePoints = []) {
    const newSetId = this.generateUniqueId(); // 生成新的练习集 ID
    const newExerciseSet = new ExerciseSet(newSetId, setName, setDescription,
knowledgePoints); // 创建练习集对象
    this.exerciseSets.push(newExerciseSet); // 将新练习集添加到管理列表
    return newExerciseSet; // 返回新创建的练习集
  },

  /**
   * 获取所有练习集
   * @returns {Array<ExerciseSet>} 练习集列表
   */
  getAllExerciseSets: function() {
    return [...this.exerciseSets]; // 返回所有练习集的副本
  },

  /**
   * 根据 ID 查找练习集
   * @param {string} setId - 练习集 ID
   * @returns {ExerciseSet | null} 找到的练习集或 null
   */
  getExerciseSetById: function(setId) {
    return this.exerciseSets.find(set => set.setId === setId) || null; // 查找并返回练习集
  },

  /**
   * 更新练习集信息

```

## 动车组乘务综合学习训练平台 V1.0

```
* @param {string} setId - 练习集 ID
* @param {object} updates - 更新内容 { setName, setDescription, knowledgePoints }
* @returns {ExerciseSet | null} 更新后的练习集或 null
*/

updateExerciseSet: function(setId, updates) {
  const exerciseSet = this.getExerciseSetById(setId); // 获取需要更新的练习集
  if (exerciseSet) {
    if (updates.setName) {
      exerciseSet.setName = updates.setName; // 更新名称
    }
    if (updates.setDescription) {
      exerciseSet.setDescription = updates.setDescription; // 更新描述
    }
    if (updates.knowledgePoints) {
      exerciseSet.knowledgePoints = updates.knowledgePoints; // 更新关联知识点
    }
    exerciseSet.updatedAt = new Date(); // 更新时间
    return exerciseSet; // 返回更新后的练习集
  }
  return null; // 未找到练习集
},

/**
 * 删除练习集
 * @param {string} setId - 练习集 ID
 * @returns {boolean} 是否删除成功
 */
deleteExerciseSet: function(setId) {
  const initialLength = this.exerciseSets.length; // 初始长度

  this.exerciseSets = this.exerciseSets.filter(set => set.setId !== setId);
// 过滤掉要删除的练习集
  return this.exerciseSets.length < initialLength; // 判断是否删除成功
},

/**
 * 添加试题到待选区
 * @param {object} questionData - 试题数据
 * @param {string} questionData.type - 试题类型 (使用 platformDataStructures.
questionTypes)
 * @param {string} questionData.content - 试题内容

 * @param {string} questionData.difficulty - 试题难度 (使用 platformDataStructures.
questionDifficulties)
 * @param {Array<string>} questionData.relatedKnowledgePoints - 关联的知识点 ID
 * @param {Array<string>} [questionData.options] - 选项 (单选/多选)
 * @param {string|boolean|Array<string>} questionData.correctAnswer - 正确答案
 * @param {string} [questionData.analysis] - 答案解析
 * @returns {QuestionItem} 创建的试题对象
```



```
*/  
addQuestionToStore: function(questionData) {  
    const questionId = this.generateUniqueId(); // 生成试题 ID  
    let newQuestion;  
    switch (questionData.type) {  
        case platformDataStructures.questionTypes.SINGLE_CHOICE:  
            newQuestion = new SingleChoiceQuestion(  
                questionId,  
                questionData.content,  
                questionData.options,  
                questionData.correctAnswer,  
                questionData.difficulty,  
                questionData.relatedKnowledgePoints  
            );  
            break;  
        case platformDataStructures.questionTypes.MULTIPLE_CHOICE:  
            newQuestion = new MultipleChoiceQuestion(  
                questionId,  
                questionData.content,  
                questionData.options,  
                questionData.correctAnswer,  
                questionData.difficulty,  
                questionData.relatedKnowledgePoints  
            );  
            break;  
        case platformDataStructures.questionTypes.TRUE_FALSE:  
            newQuestion = new TrueFalseQuestion(  
                questionId,  
                questionData.content,  
                questionData.correctAnswer,  
                questionData.difficulty,  
                questionData.relatedKnowledgePoints  
            );  
            break;  
        case platformDataStructures.questionTypes.FILL_IN_BLANK:  
            newQuestion = new FillInBlankQuestion(  
                questionId,  
                questionData.content,  
                questionData.correctAnswer,  
                questionData.difficulty,  
                questionData.relatedKnowledgePoints  
            );  
            break;  
        case platformDataStructures.questionTypes.SHORT_ANSWER:  
            newQuestion = new ShortAnswerQuestion(  
                questionId,  
                questionData.content,  
                questionData.correctAnswer,
```

```
        questionData.difficulty,
        questionData.relatedKnowledgePoints
    );
    break;
default:
    throw new Error('不支持的试题类型'); // 抛出不支持的类型错误
}
if (questionData.analysis) {
    newQuestion.analysis = questionData.analysis; // 设置答案解析
}
this.questionsDataStore.push(newQuestion); // 将新试题添加到暂存区
return newQuestion; // 返回新创建的试题
},

/**
 * 从试题暂存区获取试题
 * @param {object} filters - 过滤条件 { type, difficulty, knowledgePoints }
 * @returns {Array<QuestionItem>} 符合条件的试题列表
 */
getQuestionsFromStore: function(filters = {}) {
    return this.questionsDataStore.filter(question => {
        // 通过类型过滤
        if (filters.type && question.type !== filters.type) {
            return false;
        }
        // 通过难度过滤
        if (filters.difficulty && question.difficulty !== filters.difficulty) {
            return false;
        }
        // 通过知识点过滤 (任意一个知识点匹配即可)
        if (filters.knowledgePoints && filters.knowledgePoints.length > 0) {
            const hasMatch = question.relatedKnowledgePoints.some(kp => filters.
knowledgePoints.includes(kp));
            if (!hasMatch) {
                return false;
            }
        }
        return true; // 符合所有条件
    });
},

/**
 * 将试题添加到练习集
 * @param {string} setId - 练习集 ID
 * @param {string} questionId - 试题 ID
 * @returns {boolean} 是否添加成功
 */
addQuestionToExerciseSet: function(setId, questionId) {
```

## 动车组乘务综合学习训练平台 V1.0

```
const exerciseSet = this.getExerciseSetById(setId); // 获取练习集
const question = this.questionsDataStore.find(q => q.questionId === questionId);
// 获取试题
if (exerciseSet && question && !exerciseSet.questions.includes(questionId)) {
  exerciseSet.questions.push(questionId); // 将试题 ID 添加到练习集的试题列表
  exerciseSet.updatedAt = new Date(); // 更新时间
  return true; // 添加成功
}
return false; // 添加失败
},

/**
 * 从练习集移除试题
 * @param {string} setId - 练习集 ID
 * @param {string} questionId - 试题 ID
 * @returns {boolean} 是否移除成功
 */
removeQuestionFromExerciseSet: function(setId, questionId) {
  const exerciseSet = this.getExerciseSetById(setId); // 获取练习集
  if (exerciseSet) {
    const initialLength = exerciseSet.questions.length; // 初始长度
    exerciseSet.questions = exerciseSet.questions.filter(qId => qId !== questionId)
; // 过滤掉要移除的试题
    exerciseSet.updatedAt = new Date(); // 更新时间
    return exerciseSet.questions.length < initialLength; // 判断是否移除成功
  }
  return false; // 未找到练习集
},

/**
 * 获取练习集中包含的完整试题对象
 * @param {string} setId - 练习集 ID
 * @returns {Array<QuestionItem>} 练习集中的试题对象列表
 */
getQuestionsInExerciseSet: function(setId) {
  const exerciseSet = this.getExerciseSetById(setId); // 获取练习集
  if (exerciseSet) {
    return exerciseSet.questions
      .map(qId => this.questionsDataStore.find(q => q.questionId === qId))
      .filter(q => q !== undefined); // 将试题 ID 转换为完整的试题对象
  }
  return []; // 未找到练习集或无试题
},

/**
 * 根据条件自动组卷，将符合条件的试题添加到指定练习集
 * @param {string} setId - 目标练习集 ID
 * @param {object} criteria - 组卷条件
 * @param {object} [criteria.questionTypeCounts] - 不同题目的数量要求，例如 {
```

## 动车组乘务综合学习训练平台 V1.0

```
'singleChoice': 10, 'multipleChoice': 5 }
* @param {Array<string>} [criteria.knowledgePoints] - 知识点范围, 只从这些知识点中选择
* @param {string} [criteria.difficulty] - 难度范围, 例如 'easy', 'medium', 'hard' (
或'any'表示所有)
* @returns {boolean} 是否成功组卷
*/
autoPopulateExerciseSet: function(setId, criteria) {
  const exerciseSet = this.getExerciseSetById(setId); // 获取目标练习集
  if (!exerciseSet) {
    return false; // 未找到练习集
  }

  let availableQuestions = this.questionsDataStore; // 所有可用试题

  // 根据知识点过滤
  if (criteria.knowledgePoints && criteria.knowledgePoints.length > 0) {
    availableQuestions = availableQuestions.filter(q =>

      q.relatedKnowledgePoints.some(kp => criteria.knowledgePoints.includes(kp))
    );
  }

  // 根据难度过滤
  if (criteria.difficulty && criteria.difficulty !== 'any') {
    availableQuestions = availableQuestions.filter(q => q.difficulty === criteria.
difficulty);
  }

  const selectedQuestionIds = new Set(exerciseSet.questions); // 已添加到练习集中的试题 ID

  // 根据题型和数量要求进行选择
  if (criteria.questionTypeCounts) {
    for (const type in criteria.questionTypeCounts) {
      if (criteria.questionTypeCounts.hasOwnProperty(type)) {

        const countNeeded = criteria.questionTypeCounts[type]; // 需要的数量
        let currentCount = 0; // 当前已选数量
        // 统计现有练习集中该题型的数量, 避免重复添加
        exerciseSet.questions.forEach(qId => {
          const question = this.questionsDataStore.find(q => q.
questionId === qId);
          if (question && question.type === type) {
            currentCount++;
          }
        });

        const questionsOfType = availableQuestions.filter(q => q.
```

```
type === type && !selectedQuestionIds.has(q.questionId)); // 筛选出该题型且未添加的试题
```

```
    // 随机选取所需数量的题目
    const questionsToAdd = this.getRandomQuestions(questionsOfType,
countNeeded - currentCount); // 获取随机题目

    questionsToAdd.forEach(q => {
        selectedQuestionIds.add(q.questionId); // 添加到已选集合
    });
}
} else { // 如果没有指定题型数量，则考虑添加所有符合条件的且未添加的题目
    availableQuestions.forEach(q => {
        if (!selectedQuestionIds.has(q.questionId)) {
            selectedQuestionIds.add(q.questionId);
        }
    });
}

exerciseSet.questions = Array.from(selectedQuestionIds); // 更新练习集中的试题
exerciseSet.updatedAt = new Date(); // 更新时间
return true; // 组卷成功
},
```

```
/**
 * 从数组中随机选取指定数量的元素
 * @param {Array<any>} arr - 参与选择的数组
 * @param {number} num - 需要选择的数量
 * @returns {Array<any>} 随机选择的元素数组
 */

getRandomQuestions: function(arr, num) {
    const shuffled = [...arr].sort(() => 0.5 - Math.random()); // 随机打乱数组
    return shuffled.slice(0, num); // 返回前 num 个元素

},
};
```

```
/**
 * 知识点管理服务
 * 实际中这些数据应该从后端获取或持久化
 */
const knowledgePointService = {
    // 预定义的知识点分类，实际应从数据库加载
    knowledgeCategories: platformDataStructures.knowledgeCategories,

    /**
     * 获取所有主要知识点分类
```

## 动车组乘务综合学习训练平台 V1.0

```
* @returns {Array<object>} 知识点分类列表
*/
getAllKnowledgeCategories: function() {
  return [...this.knowledgeCategories]; // 返回所有知识点分类的副本
},

/**
 * 根据 ID 查找知识点分类
 * @param {string} categoryId - 知识点分类 ID
 * @returns {object | null} 找到的知识点分类或 null
 */
getKnowledgeCategoryById: function(categoryId) {
  return this.knowledgeCategories.find(cat => cat.id === categoryId) || null;
// 查找并返回知识点分类
},

/**
 * 添加新的知识点分类
 * @param {string} categoryName - 知识点分类名称
 * @returns {object} 新增的知识点分类对象
 */
addKnowledgeCategory: function(categoryName) {
  const newId = exerciseSetService.generateUniqueId(); // 生成唯一 ID
  const newCategory = { id: newId, name: categoryName }; // 创建新的分类对象
  this.knowledgeCategories.push(newCategory); // 添加到分类列表
  return newCategory; // 返回新分类
},

/**
 * 更新知识点分类名称
 * @param {string} categoryId - 知识点分类 ID
 * @param {string} newName - 新名称
 * @returns {object | null} 更新后的知识点分类或 null
 */
updateKnowledgeCategory: function(categoryId, newName) {
  const category = this.getKnowledgeCategoryById(categoryId); // 获取分类
  if (category) {
    category.name = newName; // 更新名称
    return category; // 返回更新后的分类
  }
  return null; // 未找到分类
},

/**
 * 删除知识点分类
 * @param {string} categoryId - 知识点分类 ID
 * @returns {boolean} 是否删除成功
 */
deleteKnowledgeCategory: function(categoryId) {
  const initialLength = this.knowledgeCategories.length; // 初始长度
```

## 动车组乘务综合学习训练平台 V1.0

```
        this.knowledgeCategories = this.knowledgeCategories.filter(cat => cat.  
id != categoryId); // 过滤掉要删除的分类  
        return this.knowledgeCategories.length < initialLength; // 判断是否删除成功  
  
    },  
};
```

Ended

```
import java.util.List;  
import java.util.ArrayList;  
import java.util.Date;  
import java.util.Map;  
import java.util.HashMap;  
  
/**  
 * 教学管理模块 - 练习集创建服务  
 * 负责练习集的创建、题目添加、属性设置等核心逻辑  
  
 */  
public class ExerciseSetCreationService {  
  
    /**  
     * 练习集的数据模型  
     */  
    public static class ExerciseSet {  
        private String setId; // 练习集唯一标识  
        private String setName; // 练习集名称  
        private String description; // 练习集描述  
        private String creatorId; // 创建者用户 ID  
        private Date creationDate; // 创建日期  
        private List<Question> questions; // 包含的题目列表  
        private String category; // 练习集分类, 例如: 安全操作, 规章制度, 设备故障  
  
        private int totalScore; // 练习集总分, 由题目分数累加  
        private boolean published; // 是否已发布  
        private Date lastModifiedDate; // 最后修改日期  
  
        public ExerciseSet(String setId, String setName, String creatorId,  
String category) {  
            this.setId = setId;  
            this.setName = setName;  
            this.creatorId = creatorId;  
            this.category = category;  
            this.creationDate = new Date();  
        }  
    }  
}
```

```
        this.questions = new ArrayList<>();

        this.totalScore = 0;
        this.published = false;
        this.lastModifiedDate = new Date();
    }

    // 获取练习集 ID
    public String getSetId() {
        return setId;
    }

    // 设置练习集名称
    public void setSetName(String setName) {
        this.setName = setName;
        this.lastModifiedDate = new Date();
    }

    // 获取练习集名称
    public String getSetName() {
        return setName;
    }

    // 设置描述
    public void setDescription(String description) {
        this.description = description;
        this.lastModifiedDate = new Date();
    }

    // 获取描述
    public String getDescription() {
        return description;
    }

    // 获取创作者 ID
    public String getCreatorId() {
        return creatorId;
    }

    // 获取创建日期
    public Date getCreationDate() {
        return creationDate;
    }

    // 获取题目列表
    public List<Question> getQuestions() {
        return questions;
    }

    // 添加题目
```



```
public void addQuestion(Question question) {
    if (question != null) {
        this.questions.add(question);
        this.totalScore += question.getScore();
        this.lastModifiedDate = new Date();
    }
}

// 移除题目
public boolean removeQuestion(String questionId) {
    boolean removed = false;
    for (int i = 0; i < questions.size(); i++) {
        if (questions.get(i).getQuestionId().equals(questionId)) {
            this.totalScore -= questions.get(i).getScore();
            questions.remove(i);
            removed = true;
            this.lastModifiedDate = new Date();
            break;
        }
    }
    return removed;
}

// 获取分类
public String getCategory() {
    return category;
}

// 设置分类
public void setCategory(String category) {
    this.category = category;
    this.lastModifiedDate = new Date();
}

// 获取总分
public int getTotalScore() {
    return totalScore;
}

// 设置总分 (通常由内部计算, 此方法用于特殊更正)
public void setTotalScore(int totalScore) {
    this.totalScore = totalScore;
    this.lastModifiedDate = new Date();
}

// 标记为已发布
public void publish() {
    this.published = true;
}
```

```
        this.lastModifiedDate = new Date();
    }

    // 标记为未发布
    public void unpublish() {
        this.published = false;
        this.lastModifiedDate = new Date();
    }

    // 判断是否已发布
    public boolean isPublished() {
        return published;
    }

    // 获取最后修改日期
    public Date getLastModifiedDate() {
        return lastModifiedDate;
    }

    // 根据 ID 查找题目
    public Question findQuestionById(String questionId) {
        for (Question q : questions) {
            if (q.getQuestionId().equals(questionId)) {
                return q;
            }
        }
        return null;
    }
}

/**
 * 题目的数据模型
 */
public static class Question {
    private String questionId; // 题目唯一标识
    private String questionText; // 题干
    private String questionType; // 题目类型 (例如: 单选、多选、判断、填空、简答)

    private List<String> options; // 选项列表 (单选、多选用)
    private List<String> correctAnswers; // 正确答案列表
    private int score; // 题目分数
    private String analysis; // 题目解析
    private String knowledgePoint; // 知识点标签

    public Question(String questionId, String questionText, String questionType,
int score) {
        this.questionId = questionId;
        this.questionText = questionText;
        this.questionType = questionType;
        this.score = score;
    }
}
```

```
        this.options = new ArrayList<>();
        this.correctAnswers = new ArrayList<>();
    }

    // 获取题目 ID
    public String getQuestionId() {
        return questionId;
    }

    // 获取题干
    public String getQuestionText() {
        return questionText;
    }

    // 设置题干
    public void setQuestionText(String questionText) {
        this.questionText = questionText;
    }

    // 获取题目类型
    public String getQuestionType() {
        return questionType;
    }

    // 设置题目类型
    public void setQuestionType(String questionType) {
        this.questionType = questionType;
    }

    // 获取选项列表
    public List<String> getOptions() {
        return options;
    }

    // 添加选项
    public void addOption(String option) {
        if (option != null && !option.trim().isEmpty() && !options.contains(
option)) {
            this.options.add(option);
        }
    }

    // 移除选项
    public boolean removeOption(String option) {
        return this.options.remove(option);
    }

    // 清除所有选项
```

```
public void clearOptions() {
    this.options.clear();
}

// 获取正确答案列表
public List<String> getCorrectAnswers() {
    return correctAnswers;
}

// 添加正确答案
public void addCorrectAnswer(String answer) {
    if (answer != null && !answer.trim().isEmpty() && !correctAnswers.
contains(answer)) {
        this.correctAnswers.add(answer);
    }
}

// 移除正确答案
public boolean removeCorrectAnswer(String answer) {
    return this.correctAnswers.remove(answer);
}

// 清除所有正确答案
public void clearCorrectAnswers() {
    this.correctAnswers.clear();
}

// 获取分数
public int getScore() {
    return score;
}

// 设置分数
public void setScore(int score) {
    if (score >= 0) {
        this.score = score;
    }
}

// 获取解析
public String getAnalysis() {
    return analysis;
}

// 设置解析
public void setAnalysis(String analysis) {
    this.analysis = analysis;
}
```

## 动车组乘务综合学习训练平台 V1.0

```
// 获取知识点
public String getKnowledgePoint() {
    return knowledgePoint;
}

// 设置知识点
public void setKnowledgePoint(String knowledgePoint) {
    this.knowledgePoint = knowledgePoint;
}
}

private Map<String, ExerciseSet> exerciseSets; // 存储所有练习集

public ExerciseSetCreationService() {
    this.exerciseSets = new HashMap<>();
}

/**
 * 创建一个新的练习集
 * @param setId 练习集唯一标识
 * @param setName 练习集名称
 * @param creatorId 创建者用户 ID
 * @param category 练习集分类
 * @return 创建成功的练习集对象，如果 ID 已存在则返回 null
 */
public ExerciseSet createExerciseSet(String setId, String setName,
String creatorId, String category) {

    if (exerciseSets.containsKey(setId)) {
        System.out.println("错误：练习集 ID " + setId + " 已存在。");
        return null;
    }
    ExerciseSet newSet = new ExerciseSet(setId, setName, creatorId, category);
    exerciseSets.put(setId, newSet);
    System.out.println("成功创建练习集: " + setName + " (ID: " + setId + ")");
    return newSet;
}

/**
 * 根据 ID 获取练习集
 * @param setId 练习集 ID
 * @return 对应的练习集对象，如果不存在则返回 null
 */
public ExerciseSet getExerciseSetById(String setId) {
    return exerciseSets.get(setId);
}

/**
```

## 动车组乘务综合学习训练平台 V1.0

```
* 更新练习集的基本属性
* @param setId 练习集 ID
* @param newName 新名称
* @param newDescription 新描述
* @param newCategory 新分类
* @return true 如果更新成功, false 如果练习集不存在或无法更新
*/
```

```
public boolean updateExerciseSetProperties(String setId, String newName,
String newDescription, String newCategory) {
    ExerciseSet exerciseSet = getExerciseSetById(setId);

    if (exerciseSet == null) {
        System.out.println("错误: 练习集 " + setId + " 不存在。");
        return false;
    }
    if (exerciseSet.isPublished()) {
        System.out.println("错误: 已发布练习集 " + setId + " 无法修改基本属性。请先取消发布。");
        return false;
    }
    if (newName != null && !newName.trim().isEmpty()) {
        exerciseSet.setSetName(newName);
    }
    if (newDescription != null) { // 允许设置为空字符串或 null 来清除描述
        exerciseSet.setDescription(newDescription);
    }
    if (newCategory != null && !newCategory.trim().isEmpty()) {
        exerciseSet.setCategory(newCategory);
    }
    System.out.println("成功更新练习集 " + setId + " 的属性。");
    return true;
}
```

```
/**
* 向指定练习集添加一道题目
* @param setId 练习集 ID
* @param question 待添加的题目对象
* @return true 如果添加成功, false 如果练习集不存在或题目已存在或无法添加
*/
public boolean addQuestionToExerciseSet(String setId, Question question) {

    ExerciseSet exerciseSet = getExerciseSetById(setId);
    if (exerciseSet == null) {
        System.out.println("错误: 练习集 " + setId + " 不存在。");
        return false;
    }
    if (exerciseSet.isPublished()) {
        System.out.println("错误: 已发布练习集 " + setId + " 无法添加题目。请先取消发布。");
        return false;
    }
}
```

```

    }
    if (exerciseSet.findQuestionById(question.getQuestionId()) != null) {
        System.out.println("错误: 题目 " + question.getQuestionId()
+ " 已存在于练习集 " + setId + " 中。");

        return false;
    }
    exerciseSet.addQuestion(question);
    System.out.println("成功将题目 " + question.getQuestionId()
+ " 添加到练习集 " + setId + "。");
    return true;
}

/**
 * 从指定练习集移除一道题目
 * @param setId 练习集 ID
 * @param questionId 待移除的题目 ID
 * @return true 如果移除成功, false 如果练习集或题目不存在或无法移除
 */
public boolean removeQuestionFromExerciseSet(String setId, String questionId) {
    ExerciseSet exerciseSet = getExerciseSetById(setId);
    if (exerciseSet == null) {
        System.out.println("错误: 练习集 " + setId + " 不存在。");
        return false;
    }
    if (exerciseSet.isPublished()) {
        System.out.println("错误: 已发布练习集 " + setId + " 无法移除题目。请先取消发布。");
        return false;
    }

    if (exerciseSet.removeQuestion(questionId)) {
        System.out.println("成功从练习集 " + setId + " 移除题目 " + questionId + "。");
        return true;
    } else {
        System.out.println("错误: 题目 " + questionId + " 不存在于练习集 " + setId + " 中。");
        return false;
    }
}

/**
 * 更新练习集中某个题目的属性
 * @param setId 练习集 ID
 * @param questionId 题目 ID
 * @param newQuestionText 新题干, 可为 null 不更新
 * @param newScore 新分数, -1 表示不更新
 * @param newAnalysis 新解析, 可为 null 不更新
 * @param newKnowledgePoint 新知识点, 可为 null 不更新
 * @return true 表示更新成功, false 表示练习集或题目不存在

```

```
*/
public boolean updateQuestionInExerciseSet(String setId, String questionId,
                                           String newQuestionText, int newScore,
                                           String newAnalysis,
String newKnowledgePoint) {
    ExerciseSet exerciseSet = getExerciseSetById(setId);
    if (exerciseSet == null) {
        System.out.println("错误: 练习集 " + setId + " 不存在。");
        return false;
    }
    if (exerciseSet.isPublished()) {
        System.out.println("错误: 已发布练习集 " + setId + " 无法修改题目。请先取消发布。");
        return false;
    }
    Question question = exerciseSet.findQuestionById(questionId);
    if (question == null) {
        System.out.println("错误: 题目 " + questionId + " 不存在于练习集 " + setId + " 中。");
        return false;
    }

    if (newQuestionText != null) {
        question.setQuestionText(newQuestionText);
    }
    if (newScore != -1) {
        // 更新题目分数会影响练习集总分, 需要特殊处理
        int oldScore = question.getScore();
        question.setScore(newScore);
        exerciseSet.setTotalScore(exerciseSet.getTotalScore()
- oldScore + newScore); // 重新计算总分
    }
    if (newAnalysis != null) {
        question.setAnalysis(newAnalysis);
    }
    if (newKnowledgePoint != null) {
        question.setKnowledgePoint(newKnowledgePoint);
    }
    System.out.println("成功更新练习集 " + setId + " 中题目 " + questionId + " 的属性。");
    return true;
}

/**
 * 发布一个练习集
 * 发布后练习集内容将不能修改, 但可以取消发布
 * @param setId 练习集 ID
 * @return true 如果发布成功, false 如果练习集不存在或已发布
 */
public boolean publishExerciseSet(String setId) {
    ExerciseSet exerciseSet = getExerciseSetById(setId);
    if (exerciseSet == null) {
```



```
        System.out.println("错误: 练习集 " + setId + " 不存在。");
        return false;
    }
    if (exerciseSet.isPublished()) {
        System.out.println("错误: 练习集 " + setId + " 已经发布。");
        return false;
    }
    if (exerciseSet.getQuestions().isEmpty()) {
        System.out.println("错误: 练习集 " + setId + " 中没有题目, 无法发布。");
        return false;
    }
    exerciseSet.publish();
    System.out.println("成功发布练习集 " + setId + "。");
    return true;
}

/**
 * 取消发布一个练习集
 * 取消发布后练习集内容可以重新修改
 *
 * @param setId 练习集 ID
 * @return true 如果取消发布成功, false 如果练习集不存在或未发布
 */
public boolean unpublishExerciseSet(String setId) {
    ExerciseSet exerciseSet = getExerciseSetById(setId);
    if (exerciseSet == null) {
        System.out.println("错误: 练习集 " + setId + " 不存在。");
        return false;
    }
    if (!exerciseSet.isPublished()) {
        System.out.println("错误: 练习集 " + setId + " 未发布。");
        return false;
    }
    exerciseSet.unpublish();
    System.out.println("成功取消发布练习集 " + setId + "。");
    return true;
}

/**
 * 删除一个练习集
 * @param setId 练习集 ID
 * @return true 如果删除成功, false 如果练习集不存在
 */
public boolean deleteExerciseSet(String setId) {
    if (!exerciseSets.containsKey(setId)) {
        System.out.println("错误: 练习集 " + setId + " 不存在, 无法删除。");
        return false;
    }
    exerciseSets.remove(setId);
    System.out.println("成功删除练习集 " + setId + "。");
}
```

```
        return true;
    }

    /**
     * 获取所有练习集的列表
     * @return 返回当前所有练习集的列表
     */
    public List<ExerciseSet> getAllExerciseSets() {
        return new ArrayList<>(exerciseSets.values());
    }

    /**
     * 更新题目选项 (适用于单选、多选)
     * @param setId 练习集 ID
     * @param questionId 题目 ID
     * @param newOptions 新的选项列表, 会替换原有的选项
     *
     * @return true 如果更新成功, false 如果练习集或题目不存在
     */
    public boolean updateQuestionOptions(String setId, String questionId,
        List<String> newOptions) {
        ExerciseSet exerciseSet = getExerciseSetById(setId);
        if (exerciseSet == null) {
            System.out.println("错误: 练习集 " + setId + " 不存在。");
            return false;
        }
        if (exerciseSet.isPublished()) {
            System.out.println("错误: 已发布练习集 " + setId + " 无法修改题目选项。请先取消发布。");
            return false;
        }
        Question question = exerciseSet.findQuestionById(questionId);
        if (question == null) {
            System.out.println("错误: 题目 " + questionId + " 不存在于练习集 " + setId + " 中。");
            return false;
        }
        String type = question.getQuestionType();
        if (!("单选".equals(type) || "多选".equals(type))) {
            System.out.println("错误: 题目 " + questionId + " 类型为 " + type + ", 无法设置选项。");
            return false;
        }

        question.clearOptions(); // 清除原有选项
        if (newOptions != null) {
            for (String option : newOptions) {
                question.addOption(option);
            }
        }
        System.out.println("成功更新练习集 " + setId + " 中题目 " + questionId + " 的选项。");
    }
}
```

```
        return true;
    }

    /**
     * 更新题目正确答案
     * @param setId 练习集 ID
     * @param questionId 题目 ID
     * @param newCorrectAnswers 新的正确答案列表，会替换原有的正确答案
     * @return true 如果更新成功，false 如果练习集或题目不存在
     */
    public boolean updateQuestionCorrectAnswers(String setId, String questionId,
        List<String> newCorrectAnswers) {
        ExerciseSet exerciseSet = getExerciseSetById(setId);
        if (exerciseSet == null) {

            System.out.println("错误：练习集 " + setId + " 不存在。");
            return false;
        }
        if (exerciseSet.isPublished()) {
            System.out.println("错误：已发布练习集 " + setId + " 无法修改题目答案。请先取消发布。");
            return false;
        }
        Question question = exerciseSet.findQuestionById(questionId);
        if (question == null) {
            System.out.println("错误：题目 " + questionId + " 不存在于练习集 " + setId + " 中。");
            return false;
        }

        question.clearCorrectAnswers(); // 清除原有答案
        if (newCorrectAnswers != null) {
            for (String answer : newCorrectAnswers) {
                question.addCorrectAnswer(answer);
            }
        }
        System.out.println("成功更新练习集 " + setId + " 中题目 " + questionId + " 的正确答案。");
        return true;
    }

    /**
     * 检验练习集的有效性，例如是否包含题目，题目分数是否合理等
     * @param setId 练习集 ID
     * @return 错误信息列表，如果为空则表示有效
     */
    public List<String> validateExerciseSet(String setId) {
        List<String> errors = new ArrayList<>();
        ExerciseSet exerciseSet = getExerciseSetById(setId);

        if (exerciseSet == null) {
```

```

        errors.add("校验失败: 练习集 " + setId + " 不存在。");
        return errors;
    }

    if (exerciseSet.getQuestions().isEmpty()) {
        errors.add("校验失败: 练习集 " + setId + " 未包含任何题目。");
    }

    int calculatedTotalScore = 0;
    for (Question q : exerciseSet.getQuestions()) {
        if (q.getScore() <= 0) {
            errors.add("校验失败: 题目 " + q.getQuestionId() + " 的分数必须大于 0。");
        }
        if (q.getQuestionText() == null || q.getQuestionText().trim().isEmpty())
        {
            errors.add("校验失败: 题目 " + q.getQuestionId() + " 的题干不能为空。");
        }
        if (q.getQuestionType() == null || q.getQuestionType().trim().isEmpty())
        {
            errors.add("校验失败: 题目 " + q.getQuestionId() + " 的类型不能为空。");
        }

        // 根据题目类型校验选项和答案
        switch (q.getQuestionType()) {
            case "单选":
            case "多选":
                if (q.getOptions().isEmpty()) {
                    errors.add("校验失败: 题目 " + q.getQuestionId() + " (类型: " + q.
getQuestionType() + ") 必须包含选项。");
                }
                if (q.getCorrectAnswers().isEmpty()) {
                    errors.add("校验失败: 题目 " + q.getQuestionId() + " (类型: " + q.
getQuestionType() + ") 必须设置正确答案。");
                }
                // 确保正确答案在选项中
                for (String answer : q.getCorrectAnswers()) {
                    if (!q.getOptions().contains(answer)) {
                        errors.add("校验失败: 题目 " + q.getQuestionId()
+ " 的正确答案 \"" + answer + "\" 不在选项列表中。");
                    }
                }
                break;
            case "判断":
                // 判断题通常只有两个选项或隐含选项
                if (q.getCorrectAnswers().size() != 1 || (!"对".equals(q.
getCorrectAnswers().get(0)) && !"错".equals(q.getCorrectAnswers().get(0)))) {
                    errors.add("校验失败: 题目 " + q.getQuestionId() + " (类型: 判断)
的正确答案应为'对'或'错'。");
                }
                break;
        }
    }

```

```
        case "填空":
        case "简答":
            if (q.getCorrectAnswers().isEmpty()) {
                errors.add("校验失败：题目 " + q.getQuestionId() + " (类型: " + q.
getQuestionType() + ") 必须设置正确答案。");
            }
            break;
        default:
            errors.add("校验失败：题目 " + q.getQuestionId() + " 包含不支持的题目类型: " + q.
getQuestionType() + "。");
            break;
    }
    calculatedTotalScore += q.getScore();
}

if (calculatedTotalScore != exerciseSet.getTotalScore()) {
    "校验失败：练习集 " + setId + " 的总分不符，内部计算总分为 " + calculatedTotalScore + "，实际记录为 " +
exercise
Set.getTotalScore() + "。");
    "校验失败：练习集 " + setId + " 的总分不符，内部计算总分为 " + calculatedTotalScore + "，实际记录为 " +
exercise
Set.getTotalScore() + "。");
    // 考虑是否在这里修复，当前只报告错误
}

if (errors.isEmpty()) {
    System.out.println("练习集 " + setId + " 校验通过。");
} else {
    System.out.println("练习集 " + setId + " 校验失败，发现以下问题：");
    for (String error : errors) {
        System.out.println("- " + error);
    }
}

return errors;
}
}
```

-- 创建练习集主表

CREATE TABLE ExerciseSets (

set\_id INT PRIMARY KEY AUTO\_INCREMENT, -- 练习集 ID

set\_name VARCHAR(255) NOT NULL, -- 练习集名称

set\_description TEXT, -- 练习集描述

created\_by INT NOT NULL, -- 创建用户 ID

creation\_time DATETIME NOT NULL DEFAULT CURRENT\_TIMESTAMP, -- 创建时间

## 动车组乘务综合学习训练平台 V1.0

```
last_modified_by INT,          -- 最后修改用户 ID
last_modified_time DATETIME,   -- 最后修改时间
status ENUM('draft', 'published',
'archived') NOT NULL DEFAULT 'draft' COMMENT '练习集状态：草稿、已发布、已归档'
);

-- 创建练习集章节表，一个练习集可以包含多个章节，用于组织题目
CREATE TABLE ExerciseSetSections (
    section_id INT PRIMARY KEY AUTO_INCREMENT, -- 章节 ID
    set_id INT NOT NULL,                       -- 所属练习集 ID
    section_name VARCHAR(255) NOT NULL,        -- 章节名称

    section_order INT NOT NULL,                -- 章节排序
    section_description TEXT,                  -- 章节描述
    REIGN KEY (set_id) REFERENCES ExerciseSets(set_id) ON DELETE CASCADE -- 关联练习集主表，删除练习集
    时删除相关章节

    REIGN KEY (set_id) REFERENCES ExerciseSets(set_id) ON DELETE CASCADE -- 关联练习集主表，删除练习集
    时删除相关章节
);

-- 创建题目类型表，用于定义题目可能的类型（例如：单选、多选、判断、填空、问答等）
CREATE TABLE QuestionTypes (
    type_id INT PRIMARY KEY AUTO_INCREMENT, -- 类型 ID

    type_name VARCHAR(100) NOT NULL UNIQUE, -- 类型名称
    type_description TEXT                   -- 类型描述
);

-- 插入常用题目类型
INSERT INTO QuestionTypes (type_name, type_description) VALUES

('single_choice', '单项选择题'),
('multiple_choice', '多项选择题'),
('true_false', '判断题'),
('fill_in_the_blank', '填空题'),
('short_answer', '简答题'),
('essay', '论述题');

-- 创建题目主表，包含所有类型题目的通用属性
CREATE TABLE Questions (
    question_id INT PRIMARY KEY AUTO_INCREMENT, -- 题目 ID

    question_text TEXT NOT NULL,                -- 题目内容
    question_type_id INT NOT NULL,              -- 题目类型 ID
    difficulty_level ENUM('easy', 'medium', 'hard') NOT NULL DEFAULT 'medium',
-- 难度级别
    knowledge_point_id INT,                     -- 关联的知识点 ID，如果存在知识点模块
    created_by INT NOT NULL,                    -- 创建用户 ID
```

## 动车组乘务综合学习训练平台 V1.0

```
creation_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, -- 创建时间
last_modified_by INT, -- 最后修改用户 ID
last_modified_time DATETIME, -- 最后修改时间
remark TEXT, -- 题目备注

FOREIGN KEY (question_type_id) REFERENCES QuestionTypes(type_id) -- 关联题目类型表
);

-- 创建练习集题目关联表，将题目与练习集中的特定章节关联
CREATE TABLE ExerciseSetQuestions (
    set_question_id INT PRIMARY KEY AUTO_INCREMENT, -- 练习集题目关联 ID
    set_id INT NOT NULL, -- 所属练习集 ID
    section_id INT, -- 所属章节 ID (可以为空，表示不属于任何章节或属于练习集根目录)
    question_id INT NOT NULL, -- 题目 ID

    question_order INT NOT NULL, -- 题目在练习集或章节中的排序
    score DECIMAL(5,2) NOT NULL DEFAULT 1.0, -- 该题在练习集中的分值
    FOREIGN KEY (set_id) REFERENCES ExerciseSets(set_id) ON DELETE CASCADE, -- 关联练习集
    FOREIGN KEY (section_id) REFERENCES ExerciseSetSections(section_id) ON DELETE SET NULL,
    -- 关联章节
    FOREIGN KEY (section_id) REFERENCES ExerciseSetSections(section_id) ON DELETE SET NULL,
    -- 关联章节
    FOREIGN KEY (question_id) REFERENCES Questions(question_id) ON DELETE CASCADE,

    -- 关联题目
    UNIQUE (set_id, question_id) -- 确保一个练习集中的同一道题目只出现一次
);

-- 创建单项选择题选项表
CREATE TABLE SingleChoiceOptions (
    option_id INT PRIMARY KEY AUTO_INCREMENT, -- 选项 ID

    question_id INT NOT NULL, -- 所属题目 ID
    option_text TEXT NOT NULL, -- 选项内容
    is_correct BOOLEAN NOT NULL, -- 是否为正确选项
    FOREIGN KEY (question_id) REFERENCES Questions(question_id) ON DELETE CASCADE -- 关联题目
);

-- 创建多项选择题选项表
CREATE TABLE MultipleChoiceOptions (
    option_id INT PRIMARY KEY AUTO_INCREMENT, -- 选项 ID

    question_id INT NOT NULL, -- 所属题目 ID

    option_text TEXT NOT NULL, -- 选项内容

    is_correct BOOLEAN NOT NULL, -- 是否为正确选项
    FOREIGN KEY (question_id) REFERENCES Questions(question_id) ON DELETE CASCADE -- 关联题目
);
```

## 动车组乘务综合学习训练平台 V1.0

```
REIGN KEY (question_id) REFERENCES Questions(question_id) ON DELETE CASCADE -- 关联题目
);
```

-- 创建判断题答案表

```
CREATE TABLE TrueFalseAnswers (
    answer_id INT PRIMARY KEY AUTO_INCREMENT, -- 答案 ID
    question_id INT NOT NULL UNIQUE,          -- 所属题目 ID (判断题只有一个答案)

    is_true BOOLEAN NOT NULL,                 -- 答案: 是/否
    FO
    REIGN KEY (question_id) REFERENCES Questions(question_id) ON DELETE CASCADE -- 关联题目
);
```

-- 创建填空题答案表, 一个填空题可能包含多个空, 每个空有对应的答案

```
CREATE TABLE FillInTheBlankAnswers (
    answer_id INT PRIMARY KEY AUTO_INCREMENT, -- 答案 ID
    question_id INT NOT NULL,                 -- 所属题目 ID
    blank_order INT NOT NULL,                 -- 填空顺序
    correct_answer TEXT NOT NULL,              -- 正确答案
    is_case_sensitive BOOLEAN NOT NULL DEFAULT FALSE, -- 答案是否区分大小写
    tolerance_level DECIMAL(3,2) NOT NULL DEFAULT 0.0, -- 答案的宽容度, 例如对数字答案
    FO
    REIGN KEY (question_id) REFERENCES Questions(question_id) ON DELETE CASCADE -- 关联题目
);
```

-- 创建简答题和论述题参考答案表

```
CREATE TABLE EssayShortAnswerAnswers (

    answer_id INT PRIMARY KEY AUTO_INCREMENT, -- 答案 ID
    question_id INT NOT NULL UNIQUE,          -- 所属题目 ID (简答/论述题通常只有一个参考答案)
    reference_answer TEXT NOT NULL,            -- 参考答案内容
    keywords TEXT,                             -- 批改关键词, 用于自动或辅助批改
    FO
    REIGN KEY (question_id) REFERENCES Questions(question_id) ON DELETE CASCADE -- 关联题目
);
```

-- 索引, 用于提高查询效率

```
CREATE INDEX idx_exercisesets_createdby ON ExerciseSets (created_by);
```

```
CREATE INDEX idx_exercisesets_status ON ExerciseSets (status);
```

```
CREATE INDEX idx_exercisesetsections_setid ON ExerciseSetSections (set_id);
```

```
CREATE INDEX idx_questions_questiontypeid ON Questions (question_type_id);
```

```
CREATE INDEX idx_questions_createdby ON Questions (created_by);
```

```
CREATE INDEX idx_questions_difficultylevel ON Questions (difficulty_level);
```

```
CREATE INDEX idx_exercisesetquestions_setid ON ExerciseSetQuestions (set_id);
```

```
CREATE INDEX idx_exercisesetquestions_questionid ON ExerciseSetQuestions (question_id)
```



```
);  
CREATE INDEX idx_singlechoiceroptions_questionid ON SingleChoiceOptions (question_id);  
CREATE INDEX idx_multiplechoiceroptions_questionid ON MultipleChoiceOptions (question_id);  
  
CREATE INDEX idx_truefalseanswers_questionid ON TrueFalseAnswers (question_id);  
CREATE INDEX idx_fillintheblankanswers_questionid ON FillInTheBlankAnswers (question_id);  
CREATE INDEX idx_essayshortansweranswers_questionid ON EssayShortAnswerAnswers (question_id);
```

```
<!DOCTYPE html>  
<html lang="zh-CN">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>动车组乘务综合学习训练平台 - 讨论论坛</title>  
  <style>  
    /* 全局样式 */  
    body {  
      font-family: 'PingFang SC', 'Microsoft YaHei', sans-serif;  
      margin: 0;  
      padding: 0;  
      background-color: #f4f5f7;  
      color: #333;  
      line-height: 1.6;  
    }  
  
    .container {  
  
      max-width: 1200px;  
      margin: 20px auto;  
      padding: 20px;  
      background-color: #fff;  
      border-radius: 8px;  
      box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08); /* 页面容器阴影 */  
    }  
  
    /* 顶部导航和面包屑 */  
    header {  
      background-color: #007bff;
```

```
        color: #fff;
        padding: 15px 0;
        text-align: center;
        border-top-left-radius: 8px;
        border-top-right-radius: 8px;
        margin-bottom: 20px;
    }

    header h1 {
        margin: 0;
        font-size: 2.2em;
        letter-spacing: 1px;
        color: #e6f2ff;
    }

    .breadcrumb {
        background-color: #e9ecef;

        padding: 10px 20px;
        border-radius: 5px;
        margin-bottom: 20px;
        font-size: 0.95em;
        color: #666;
    }

    .breadcrumb a {
        text-decoration: none;
        color: #007bff;
    }

    .breadcrumb a:hover {
        text-decoration: underline;
    }

    .breadcrumb span {
        margin: 0 8px;
        color: #888;
    }

    /* 论坛内容区域 */
    .forum-header {
        display: flex;
        justify-content: space-between;

        align-items: center;
        margin-bottom: 25px;
        padding-bottom: 15px;
        border-bottom: 1px solid #eee;
    }
```

```
.forum-header h2 {  
  
    margin: 0;  
    font-size: 1.8em;  
    color: #007bff;  
}  
  
.forum-actions button {  
    background-color: #28a745;  
    color: #fff;  
    border: none;  
    padding: 10px 20px;  
    border-radius: 5px;  
    cursor: pointer;  
    font-size: 1em;  
    transition: background-color 0.3s ease;  
}  
  
.forum-actions button:hover {  
    background-color: #218838;  
}  
  
/* 帖子列表 */  
.thread-list {  
    list-style: none;  
    padding: 0;  
    margin: 0;  
}  
  
.thread-item {  
    background-color: #fff;  
    border: 1px solid #ddd;  
    border-radius: 8px;  
    margin-bottom: 15px;  
    padding: 18px 25px;  
    box-shadow: 0 1px 5px rgba(0, 0, 0, 0.05); /* 单个帖子阴影 */  
    transition: transform 0.2s ease, box-shadow 0.2s ease;  
    display: flex;  
    align-items: center;  
}  
  
.thread-item:hover {  
    transform: translateY(-3px);  
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);  
}  
  
.thread-item-icon {
```

```
        font-size: 2.5em; /* 图标大小 */
        color: #007bff;
        margin-right: 20px;
        flex-shrink: 0; /* 防止图标缩小 */
    }

    .thread-content {
        flex-grow: 1; /* 内容区占据剩余空间 */
    }

    .thread-content h3 {
        margin-top: 0;
        margin-bottom: 8px;
        font-size: 1.3em;
    }

    .thread-content h3 a {
        text-decoration: none;
        color: #333;
        transition: color 0.2s ease;
    }

    .thread-content h3 a:hover {
        color: #007bff;
    }

    .thread-meta {
        font-size: 0.9em;
        color: #777;
        display: flex;
        align-items: center;
    }

    .thread-meta span {
        margin-right: 15px;
        display: flex;
        align-items: center;
    }

    .thread-meta span i {
        margin-right: 5px;
        color: #999;
    }

    .thread-stats {
        width: 150px; /* 统计信息宽度 */

        text-align: right;
        border-left: 1px solid #eee;
```

```
padding-left: 20px;
margin-left: 20px;
flex-shrink: 0; /* 防止统计信息缩小 */
}

.thread-stats div {
    font-size: 0.9em;
    color: #666;

    margin-bottom: 5px;
}

.thread-stats .stat-value {
    font-weight: bold;
    color: #007bff;
    font-size: 1.1em;
}

/* 模拟图标库 */
.icon-comments::before { content: " "; }
.icon-eye::before { content: " "; }
.icon-user::before { content: " "; }
.icon-time::before { content: "🕒"; }
.icon-pin::before { content: " "; } /* 置顶图标 */
.icon-fire::before { content: " "; } /* 热门图标 */

/* 分页器 */
.pagination {
    text-align: center;
    margin-top: 30px;
    padding-top: 20px;
    border-top: 1px solid #eee;
}

.pagination a {
    display: inline-block;
    text-decoration: none;
    color: #007bff;
    padding: 8px 15px;
    margin: 0 5px;
    border: 1px solid #007bff;
    border-radius: 5px;
    transition: background-color 0.3s ease, color 0.3s ease;
}

.pagination a.active,
.pagination a:hover {
    background-color: #007bff;
    color: #fff;
    border-color: #007bff;
}
```

```
}

.pagination a.disabled {
  color: #ccc;
  border-color: #ccc;
  cursor: not-allowed;
  background-color: #f9f9f9;
}

/* 侧边栏和广告位 */
.main-layout {
  display: flex;
  justify-content: space-between;
  gap: 20px; /* 主布局内间距 */
}

.main-content {
  flex: 3; /* 主内容区域占据更多空间 */
}

.sidebar {
  flex: 1; /* 侧边栏占据剩余空间 */
  background-color: #fff;

  padding: 20px;
  border-radius: 8px;

  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.05);
  min-width: 280px; /* 侧边栏最小宽度 */
}

.sidebar-block {
  margin-bottom: 25px;
  padding-bottom: 20px;
  border-bottom: 1px solid #eee;
}

.sidebar-block:last-child {
  border-bottom: none;
  margin-bottom: 0;
  padding-bottom: 0;
}

.sidebar-block h3 {
  margin-top: 0;
  margin-bottom: 15px;
  font-size: 1.3em;
  color: #007bff;
  border-left: 4px solid #007bff;
```

```
padding-left: 10px;
}

.sidebar-block ul {
list-style: none;
padding: 0;
margin: 0;
}

.sidebar-block ul li {
margin-bottom: 10px;
font-size: 0.95em;
}

.sidebar-block ul li a {
text-decoration: none;
color: #555;
transition: color 0.2s ease;

display: block;
padding: 5px 0;
}

.sidebar-block ul li a:hover {
color: #007bff;
background-color: #f0f8ff;
padding-left: 5px;

border-radius: 4px;
}

.sidebar-block ul li a .icon {
margin-right: 8px;
color: #999;
}

}

.ad-banner {
text-align: center;
margin-top: 30px;
background-color: #e9ecef;
padding: 30px 20px;
border-radius: 8px;
color: #666;
font-size: 1.1em;
border: 1px dashed #ccc;
}

.ad-banner h4 {
```

```
        margin-top: 0;
        color: #333;
    }

    .ad-banner p {
        margin-bottom: 0;
    }

    /* 论坛置顶和热门标签 */
    .thread-meta .tag {
        background-color: #f0f0f0;
        color: #555;
        padding: 2px 8px;
        border-radius: 4px;
        font-size: 0.8em;
        margin-left: 10px;
        font-weight: normal;
    }

    .thread-meta .tag.sticky {
        background-color: #ffc107;
        color: #fff;
    }

    .thread-meta .tag.hot {
        background-color: #dc3545;
        color: #fff;
    }

</style>
</head>
<body>
    <div class="container">
        <!-- 头部区域，显示平台名称 -->
        <header>
            <h1>动车组乘务综合学习训练平台</h1>
        </header>

        <!-- 面包屑导航，指示当前位置 -->
        <div class="breadcrumb">
            <a href="#">首页</a> <span>/</span>
            <a href="#">交流反馈</a> <span>/</span>
            <span>讨论论坛</span>
        </div>

        <!-- 主布局，包含主要内容区和侧边栏 -->
        <div class="main-layout">
            <!-- 论坛主内容区 -->
            <div class="main-content">
```



## 动车组乘务综合学习训练平台 V1.0

```
<!-- 论坛头部，包含标题和发帖按钮 -->
<div class="forum-header">
  <h2>乘务交流论坛</h2>
  <div class="forum-actions">
    <button onclick="window.location.href='new_thread.html'">+
发布新贴</button>
  </div>

</div>

<!-- 帖子列表 -->
<ul class="thread-list">
  <!-- 单个帖子项示例 - 置顶帖 -->
  <li class="thread-item">
    <div class="thread-item-icon"><i class="icon-pin"></i></div>
    <div class="thread-content">
      【置顶】关于乘务安全操作规程的讨论和建议</a><span class="tag sticky">置顶</span><span class="tag
hot">热门<

/span></h3>
      【置顶】关于乘务安全操作规程的讨论和建议</a><span class="tag sticky">置顶</span><span class="tag
hot">热门<
/span></h3>
      <div class="thread-meta">
        <span><i class="icon-user"></i> 发布者：安全管理员</span>
        <span><i class="icon-time"></i> 发布时间：2023-10-26
10:30</span>
      </div>
    </div>
    <div class="thread-stats">
      <div>回复：<span class="stat-value">128</span></div>
      <div>浏览：<span class="stat-value">5678</span></div>
    </div>
  </li>

  <!-- 单个帖子项示例 - 热门帖 -->
  <li class="thread-item">
    <div class="thread-item-icon"><i class="icon-fire"></i></div>
    <div class="thread-content">
      <h3><a href="thread_detail.html?id=1002">
动车组应急处置案例分析与经验分享</a><span class="tag hot">热门</span></h3>
      <div class="thread-meta">
        <span><i class="icon-user"></i> 发布者：李乘务长</span>
        <span><i class="icon-time"></i> 发布时间：2023-10-25
15:45</span>
      </div>
    </div>
    <div class="thread-stats">
      <div>回复：<span class="stat-value">89</span></div>
      <div>浏览：<span class="stat-value">3210</span></div>
    </div>
  </li>
</ul>
```

```
</li>

<!-- 普通帖子项示例 -->
<li class="thread-item">
  <div class="thread-item-icon"><i class="icon-comments"></i><
/div>

  <div class="thread-content">
    <h3><a href="thread_detail.html?id=1003">新版乘务手册解读与常见问题答疑<
/a></h3>

    <div class="thread-meta">
      <span><i class="icon-user"></i> 发布者: 陈教员</span>
      <span><i class="icon-time"></i> 发布时间: 2023-10-24
09:00</span>
    </div>
  </div>
  <div class="thread-stats">

    <div>回复: <span class="stat-value">45</span></div>
    <div>浏览: <span class="stat-value">1890</span></div>
  </div>
</li>

<li class="thread-item">
  <div class="thread-item-icon"><i class="icon-comments"></i><
/div>

  <div class="thread-content">
    <h3><a href="thread_detail.html?id=1004">旅客服务突发事件应对策略讨论<
/a></h3>

    <div class="thread-meta">
      <span><i class="icon-user"></i> 发布者: 王乘务员</span>
      <span><i class="icon-time"></i> 发布时间: 2023-10-23
11:20</span>
    </div>
  </div>
  <div class="thread-stats">
    <div>回复: <span class="stat-value">62</span></div>
    <div>浏览: <span class="stat-value">2500</span></div>
  </div>
</li>

<li class="thread-item">
  <div class="thread-item-icon"><i class="icon-comments"></i><
/div>

  <div class="thread-content">
    <h3><a href="thread_detail.html?id=1005">
动车组设备故障初步判断与报修流程</a></h3>
    <div class="thread-meta">
      <span><i class="icon-user"></i> 发布者: 技术支持</span>
```

```

                <span><i class="icon-time"></i> 发布时间: 2023-10-22
14:00</span>
                </div>
            </div>
            <div class="thread-stats">
                <div>回复: <span class="stat-value">31</span></div>
                <div>浏览: <span class="stat-value">1450</span></div>
            </div>
        </li>

        <li class="thread-item">
            <div class="thread-item-icon"><i class="icon-comments"></i> <
/ div>
            <div class="thread-content">
                <h3><a href="thread_detail.html?id=1006">乘务人员心理健康与压力疏导<
/a></h3>
                <div class="thread-meta">
                    <span><i class="icon-user"></i> 发布者: 心理辅导员</span>
                    <span><i class="icon-time"></i> 发布时间: 2023-10-21
16:00</span>
                </div>
            </div>
            <div class="thread-stats">
                <div>回复: <span class="stat-value">78</span></div>
                <div>浏览: <span class="stat-value">3015</span></div>
            </div>
        </li>

        <li class="thread-item">
            <div class="thread-item-icon"><i class="icon-comments"></i> <
/ div>
            <div class="thread-content">
                <h3><a href="thread_detail.html?id=1007">关于优化站车交接班流程的建议<
/a></h3>
                <div class="thread-meta">
                    <span><i class="icon-user"></i> 发布者: 张乘务员</span>

                    <span><i class="icon-time"></i> 发布时间: 2023-10-20
09:30</span>
                </div>
            </div>
            <div class="thread-stats">
                <div>回复: <span class="stat-value">25</span></div>
                <div>浏览: <span class="stat-value">1120</span></div>

            </div>
        </li>

        <li class="thread-item">
```

```
<div class="thread-item-icon"><i class="icon-comments"></i><
/div>
<div class="thread-content">
  <h3><a href="thread_detail.html?id=1008">旅客投诉处理技巧与案例分享<
/a></h3>
  <div class="thread-meta">
    <span><i class="icon-user"></i> 发布者：服务主管</span>
    <span><i class="icon-time"></i> 发布时间：2023-10-19
13:00</span>

    </div>
  </div>
  <div class="thread-stats">
    <div>回复：<span class="stat-value">55</span></div>

    <div>浏览：<span class="stat-value">2300</span></div>
  </div>
</li>
</ul>

<!-- 分页器 -->
<div class="pagination">
  <a href="#" class="disabled">上一页</a>
  <a href="#" class="active">1</a>
  <a href="#">2</a>
  <a href="#">3</a>
  <a href="#">4</a>
  <span>...</span>
  <a href="#">10</a>
  <a href="#">下一页</a>
</div>
</div>

<!-- 侧边栏，包含热门话题和论坛分类 -->
<div class="sidebar">
  <!-- 热门话题模块 -->
  <div class="sidebar-block">
    <h3>热门话题</h3>
    <ul>
      <li><a href="#"><i class="icon">#</i>安全生产月</a></li>

      <li><a href="#"><i class="icon">#</i>服务质量提升</a></li>
      <li><a href="#"><i class="icon">#</i>应急演练复盘</a></li>
      <li><a href="#"><i class="icon">#</i>列车餐食改进</a></li>
      <li><a href="#"><i class="icon">#</i>新员培训心得</a></li>
    </ul>
  </div>

  <!-- 论坛分类模块 -->
  <div class="sidebar-block">
```

```
<h3>论坛分类</h3>
<ul>
  <li><a href="#"><i class="icon"> </i> 安全规章</a></li>
  <li><a href="#"><i class="icon"> </i> 旅客服务</a></li>
  <li><a href="#"><i class="icon"> </i> 设备操作</a></li>
  <li><a href="#"><i class="icon"> </i> 学习资料</a></li>
  <li><a href="#"><i class="icon"> </i> 经验分享</a></li>
  <li><a href="#"><i class="icon"> </i> 建议反馈</a></li>
  <li><a href="#"><i class="icon"> </i> 公告通知</a></li>
</ul>

</div>

<!-- 广告/合作宣传位 -->
<div class="ad-banner">
  <h4>学习资源推荐</h4>
  <p>高质量乘务技能课程，助您职业发展! </p>
  <p><a href="#" style="color: #007bff; text-decoration:
underline;">了解更多</a></p>
</div>
</div>
</div>
</div>
</body>
</html>
```

```
/**
```

```
 * 讨论论坛模块
```

```
 * 负责用户之间在学习平台上的交流和讨论
```

```
 */
```

```
// 讨论帖数据结构定义
```

```
class DiscussionPost {
```

```
  /**
```

```
   * @param {string} id - 帖子唯一标识
```

```
   * @param {string} title - 帖子标题
```

```
   * @param {string} content - 帖子内容
```

```
   * @param {string} authorId - 发帖用户 ID
```

```
   * @param {number} timestamp - 发帖时间戳
```

```
   * @param {Array<string>} tags - 帖子标签
```

```
   * @param {Array<string>} attachments - 附件 URL 列表
```

```
  */
```

```
  constructor(id, title, content, authorId, timestamp, tags = [], attachments = []) {
```

```
    this.id = id;
```

```
    this.title = title;
```

```
    this.content = content;
```

```
    this.authorId = authorId;
```

```
    this.timestamp = timestamp;
```

```
this.tags = tags;
this.attachments = attachments;
this.replies = []; // 回复列表
this.views = 0; // 浏览量
this.likes = 0; // 点赞数
this.isPinned = false; // 是否置顶
this.isClosed = false; // 是否关闭
}
}

// 回复数据结构定义
class PostReply {
    /**
     * @param {string} id - 回复唯一标识
     * @param {string} postId - 所属帖子 ID
     * @param {string} content - 回复内容
     * @param {string} authorId - 回复用户 ID
     * @param {number} timestamp - 回复时间戳
     * @param {string|null} parentReplyId - 父回复 ID, 用于多级回复
     */

    constructor(id, postId, content, authorId, timestamp, parentReplyId = null) {
        this.id = id;

        this.postId = postId;
        this.content = content;
        this.authorId = authorId;
        this.timestamp = timestamp;
        this.parentReplyId = parentReplyId;
        this.likes = 0; // 回复点赞数
    }
}

// 论坛服务接口
class ForumService {
    constructor() {
        this.posts = new Map(); // 存储所有讨论帖, 键为帖子 ID
        this.replies = new Map(); // 存储所有回复, 键为回复 ID
    }

    /**
     * 创建新的讨论帖
     * @param {string} authorId - 发帖用户 ID
     * @param {string} title - 帖子标题
     * @param {string} content - 帖子内容

     * @param {Array<string>} tags - 帖子标签
     * @param {Array<string>} attachments - 附件 URL 列表
    */
}
```

## 动车组乘务综合学习训练平台 V1.0

```
* @returns {DiscussionPost} 创建的讨论帖对象
*/
createPost(authorId, title, content, tags = [], attachments = []) {
  const postId = `post_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;
  const timestamp = Date.now();
  const newPost = new DiscussionPost(postId, title, content, authorId, timestamp,
tags, attachments);

  this.posts.set(postId, newPost);
  return newPost;
}

/**
 * 获取指定 ID 的讨论帖
 * 增加帖子浏览量
 * @param {string} postId - 帖子 ID
 * @returns {DiscussionPost|undefined} 讨论帖对象或 undefined
 */
getPost(postId) {

  const post = this.posts.get(postId);
  if (post) {
    post.views++; // 增加浏览量
  }
  return post;
}

/**
 * 更新讨论帖内容
 * @param {string} postId - 帖子 ID
 * @param {string} newTitle - 新标题
 * @param {string} newContent - 新内容
 * @param {Array<string>} newTags - 新标签
 * @param {Array<string>} newAttachments - 新附件 URL 列表
 * @returns {boolean} 是否更新成功
 */
updatePost(postId, newTitle, newContent, newTags, newAttachments) {
  const post = this.posts.get(postId);
  if (post) {
    post.title = newTitle;
    post.content = newContent;
    post.tags = newTags;
    post.attachments = newAttachments;
    return true;
  }
  return false;
}
```

```
/**
 * 删除讨论帖及其所有回复
 * @param {string} postId - 帖子 ID
 * @returns {boolean} 是否删除成功
 */
deletePost(postId) {
  const post = this.posts.get(postId);
  if (post) {
    // 删除所有与该帖子相关的回复
    this.replies.forEach((reply, replyId) => {
      if (reply.postId === postId) {
        this.replies.delete(replyId);
      }
    });
    this.posts.delete(postId);
    return true;
  }
  return false;
}

/**
 * 为讨论帖添加回复

 * @param {string} postId - 所属帖子 ID
 * @param {string} authorId - 回复用户 ID

 * @param {string} content - 回复内容
 * @param {string|null} parentReplyId - 父回复 ID, 用于多级回复
 * @returns {PostReply|undefined} 创建的回回复对象或 undefined
 */
addReply(postId, authorId, content, parentReplyId = null) {
  const post = this.posts.get(postId);
  if (post) {

    const replyId = `reply_${Date.now()}_${Math.random().toString(36).substr(2, 9)}
`;
    const timestamp = Date.now();
    const newReply = new PostReply(replyId, postId, content, authorId, timestamp,
parentReplyId);
    this.replies.set(replyId, newReply);
    post.replies.push(replyId); // 将回复 ID 添加到帖子回复列表中
    return newReply;
  }
  return undefined;
}

/**
 * 获取指定 ID 的回复
```



```
* @param {string} replyId - 回复 ID
* @returns {PostReply|undefined} 回复对象或 undefined
*/
getReply(replyId) {
    return this.replies.get(replyId);
}

/**
 * 删除指定 ID 的回复
 * @param {string} replyId - 回复 ID
 * @returns {boolean} 是否删除成功
 */
deleteReply(replyId) {
    const reply = this.replies.get(replyId);

    if (reply) {
        const post = this.posts.get(reply.postId);
        if (post) {
            // 从帖子回复列表中移除该回复
            post.replies = post.replies.filter(id => id !== replyId);
        }
        this.replies.delete(replyId);

        return true;
    }
    return false;
}

/**
 * 对帖子进行点赞或取消点赞
 * @param {string} postId - 帖子 ID
 * @param {boolean} isLike - true 表示点赞, false 表示取消点赞
 * @returns {boolean} 是否操作成功
 */
togglePostLike(postId, isLike) {
    const post = this.posts.get(postId);
    if (post) {
        post.likes += isLike ? 1 : -1;
        post.likes = Math.max(0, post.likes); // 确保点赞数不为负
        return true;
    }
    return false;
}

/**
 * 对回复进行点赞或取消点赞
```

## 动车组乘务综合学习训练平台 V1.0

```
* @param {string} replyId - 回复 ID
* @param {boolean} isLike - true 表示点赞, false 表示取消点赞
* @returns {boolean} 是否操作成功
*/
toggleReplyLike(replyId, isLike) {
  const reply = this.replies.get(replyId);
  if (reply) {
    reply.likes += isLike ? 1 : -1;
    reply.likes = Math.max(0, reply.likes); // 确保点赞数不为负
    return true;
  }
  return false;
}

/**
 * 获取所有讨论帖, 可根据条件过滤和排序

 * @param {Object} [filters={}] - 过滤条件, 如 { authorId: 'xxx', tag: 'yyy' }
 * @param {Object} [sort = { by: 'timestamp', order: 'desc' }] - 排序方式, 如 {
by: 'views', order: 'desc' }
 * @returns {Array<DiscussionPost>} 过滤并排序后的讨论帖列表
*/
getAllPosts(filters = {}, sort = {
  by: 'timestamp',
  order: 'desc'
}) {

  let result = Array.from(this.posts.values());

  // 应用过滤
  if (filters.authorId) {
    result = result.filter(post => post.authorId === filters.authorId);
  }
  if (filters.tag) {
    result = result.filter(post => post.tags.includes(filters.tag));
  }
  if (filters.keyword) {
    const keywordLower = filters.keyword.toLowerCase();
    result = result.filter(post =>
      post.title.toLowerCase().includes(keywordLower) ||
      post.content.toLowerCase().includes(keywordLower)
    );
  }

  // 应用排序
  result.sort((a, b) => {
    let valA = a[sort.by];
    let valB = b[sort.by];
```

```
        if (typeof valA === 'string') {
            valA = valA.toLowerCase();
            valB = valB.toLowerCase();
        }

        if (sort.order === 'asc') {
            return valA > valB ? 1 : -1;
        } else { // desc
            return valB > valA ? 1 : -1;
        }
    });

    return result;
}

/**
 * 获取指定讨论帖的所有回复，并按时间排序
 * @param {string} postId - 帖子 ID
 * @returns {Array<PostReply>} 该帖子的所有回复列表
 */
getRepliesForPost(postId) {
    const post = this.posts.get(postId);
    if (!post) {

        return [];
    }
    // 获取所有回复对象并按时间戳升序排序
    return post.replies
        .map(replyId => this.replies.get(replyId))
        .filter(reply => reply !== undefined)
        .sort((a, b) => a.timestamp - b.timestamp);
}

/**
 * 置顶或取消置顶讨论帖
 * 只有管理员或版主可以操作
 * @param {string} postId - 帖子 ID
 * @param {boolean} isPinned - true 表示置顶，false 表示取消置顶
 * @returns {boolean} 是否操作成功
 */
setPostPinnedStatus(postId, isPinned) {
    const post = this.posts.get(postId);
    if (post) {
        post.isPinned = isPinned;
        return true;
    }
}
```

```
        return false;
    }

    /**
     * 关闭或开放讨论帖，关闭后不能回复
     * 只有管理员或版主可以操作
     * @param {string} postId - 帖子 ID
     * @param {boolean} isClosed - true 表示关闭，false 表示开放
     * @returns {boolean} 是否操作成功
     */
    setPostClosedStatus(postId, isClosed) {
        const post = this.posts.get(postId);
        if (post) {
            post.isClosed = isClosed;

            return true;
        }
        return false;
    }
}
```